

Testing

Contents

Foundations

1. [Why we test](#)
2. [The test pyramid, and its critics](#)
3. [Test levels and test purposes](#)
4. [What makes a good test](#)
5. [Deciding what to test](#)

Writing a unit test

6. [What a unit is](#)
7. [Structure and naming](#)
8. [Parameterized and property-based tests](#)

Test doubles

9. [The five kinds of double](#)
10. [The two schools](#)
11. [When not to use a double](#)

Beyond the unit

12. [Integration tests](#)
13. [Contract testing](#)
14. [End to end tests](#)

Practices

15. [Test driven development](#)
16. [Testing time and asynchronous code](#)
17. [Test data](#)
18. [Working with legacy code](#)

Judging the tests

19. [Coverage](#)
20. [Mutation testing](#)
21. [Flaky tests](#)

Reference

- 22. [Interview questions](#)
- 23. [Common mistakes](#)
- 24. [Quick reference](#)

1. Why we test

The weak answer is "to catch bugs". Tests do catch bugs, but if that were the whole reason we would run them once and delete them.

Tests are what make code safe to change. A suite is a written statement of what the system must keep doing. Without one, every change is a gamble, and the rational response to a gamble is to avoid taking it, which is how a codebase becomes something nobody will touch.

Three things a good suite provides:

- **Confidence to change.** Refactoring is only possible when something reliable tells us we broke it. Without tests, "refactoring" is rewriting and hoping.
- **An executable description, checked by the build.** Prose documentation drifts away from the code silently and nothing reports it. A test that no longer matches the code fails. It can still go stale in the other direction, when the requirement changes and nobody updates the test, so this catches drift in the code rather than drift in our understanding of what the code should do.
- **Design pressure.** Code that is hard to test is usually hard to use. If a test needs six doubles, the class has six dependencies, and the test is reporting a design problem rather than a testing problem.

The third one is the answer most candidates miss, and it is the one an interviewer notices, because it treats tests as feedback about the design rather than as a chore performed afterwards.

What tests do not provide is proof. Testing can show that faults are present; it cannot show that none remain. The observation is Dijkstra's, and it is worth knowing because it sets the honest limit on what any suite is claiming.

2. The test pyramid, and its critics

Mike Cohn's shape: a wide base of fast unit tests, fewer integration tests above them, and very few end to end tests at the top.

Level	Speed	What it catches	What it costs
Unit	Fastest	Logic errors in one piece of behaviour	Little. Runs anywhere, needs nothing
Integration	Slower	Wiring, mapping, queries, serialisation	A container or a running dependency

Level	Speed	What it catches	What it costs
End to end	Slowest	Whole journeys, and configuration errors nothing else sees	A deployed environment, and constant maintenance

Read that middle column as a general ordering rather than a definition. A large unit suite takes minutes, a well-built integration test can be quick, and speed is a consequence of what a test needs rather than the thing that decides which level it belongs to.

The reasoning is about cost and feedback, not virtue. A unit test fails in milliseconds and names one class. An end to end test fails in minutes and reports that something, somewhere, is wrong.

The inverted shape, many end to end tests over a thin layer of unit tests, is common enough to have a name of its own, the **ice cream cone**. It tends toward slow suites, weak failure isolation, and eventually enough flakiness that nobody believes a red build until they have re-run it.

The pyramid is a heuristic, not a law, and two criticisms of it are fair:

- **Its layers are undefined.** "Unit" and "integration" mean different things in different teams, so the pyramid recommends more of something it has not specified. Section 6 and section 12 are about pinning those two words down.
- **The right shape depends on the system.** Kent C. Dodds' **testing trophy** makes integration the largest layer, on the argument that it buys the most confidence per unit of cost. For a service that is mostly orchestration, with the real complexity in how components combine rather than inside any one of them, that is straightforwardly true: there is little algorithmic logic to unit test.

What survives both readings is the principle the pyramid was reaching for: **use the fastest test that can catch the fault**. Push a check down a level when it can be caught there, and accept the expensive level only for what genuinely needs it.

3. Test levels and test purposes

These are two different dimensions, and confusing them is common enough that separating them cleanly is worth marks on its own.

Level describes how much of the system a test runs: unit, integration, contract, end to end. It is what the pyramid is about.

Purpose describes why the test exists:

Purpose	What it is for
Regression	Confirming that behaviour which previously worked still works. Most of a mature suite is this
Smoke	A quick check that a build or environment is usable at all, run before anything more expensive
Acceptance	Confirming behaviour against what the business asked for, often expressed in their language

Purpose	What it is for
Performance	Behaviour under load. A different activity, covered in the observability note
Security	Authorisation, injection, and handling of untrusted input

The two dimensions are independent. A smoke test is often end to end, but a smoke test can equally be a single unit test on the one calculation everything depends on. A regression test can sit at any level, and usually the correct level is the lowest one that would have caught the fault. An acceptance test is frequently an integration test rather than an end to end one.

Which is why "we need more integration tests" and "we need more regression tests" are answers to different questions, and why an interviewer asking about one and hearing about the other notices.

4. What makes a good test

Properties, in roughly descending order of how much they matter:

- **It fails when the behaviour breaks.** A test that passes regardless is worse than no test, because it reports safety that does not exist.
- **It fails for one reason.** Then a failure tells us where to look.
- **It does not fail otherwise.** Flakiness destroys a suite's value faster than missing tests do, for reasons section 21 goes into.
- **It reads as a description of the behaviour,** because the test is the documentation somebody will actually read.
- **It is fast.** A suite that is slow stops being run, and a suite that is not run is not a suite.
- **It is independent.** Order must not matter, and no test may depend on another having run first.

The property that produces the most argument, and the most valuable one to be able to defend:

Test behaviour, not implementation

A test coupled to **how** the code works fails every time the code is restructured, even when nothing observable changed. That trains a team to treat failures as noise, which is the point at which the suite stops protecting anything.

A test coupled to **what** the code does fails only when the behaviour changed, which is exactly the signal we wanted.

The practical rule: **if a pure refactoring with no change in behaviour breaks the test, the test was testing the wrong thing.**

5. Deciding what to test

The unstated assumption behind most testing advice is that more tests are better. They are not free: every test is written once, run thousands of times, maintained forever, and adds friction to every future change. So the question is not whether to test but **where to spend the effort**, and having an answer is what separates a senior response from a thorough one.

Test depth should follow risk, and risk here has five components worth naming:

Factor	The question it asks
Business impact	What happens if this is wrong? Charging the wrong amount is not the same as a misaligned label
Complexity	How many branches, boundaries and interactions does it have? Simple code fails in fewer ways
Change frequency	Code touched weekly earns tests; code untouched for three years mostly earns being left alone
Reach	How much depends on it? A shared calculation is worth more than one screen
Uncertainty	How confident are we that we understood the requirement? Tests are cheapest where we are least sure

Applied, that produces very different answers within one service:

```
Payment and discount calculation
  high impact, high complexity, shared
  -> unit tests over every branch and boundary, integration for persistence,
      one end to end for the whole purchase

Order status transitions
  moderate impact, real rules, changes often
  -> unit tests for the legal and illegal transitions

Mapping an entity to a response object, no logic
  low impact, no branches
  -> covered incidentally by the controller test. No dedicated test
```

What not to test follows from the same reasoning, and it is a fair interview question because answering it well shows judgement rather than diligence:

- **Getters, setters and plain data holders.** No behaviour, so nothing to assert.
- **Straight-through mapping with no logic.** An integration test covers it as a side effect; a dedicated test restates the code.
- **The framework or the library.** Testing that a query annotation works is testing somebody else's product. Test our query against a real database instead, which is a different thing.

- **Private methods.** Test through the public surface. A private method that feels like it needs its own test is usually a separate class trying to exist.
- **Generated code**, unless we wrote the generator.
- **Combinations already covered by a property.** Once a property-based test asserts an invariant over generated inputs, adding examples of the same invariant adds maintenance without adding information.

The general form of the answer: **test behaviour that can be wrong in a way that matters.** Everything else is cost.

6. What a unit is

The most contested word in testing, and a fair interview question precisely because there is no settled answer.

Two positions:

- **A unit is a class or a method.** Every collaborator is replaced with a double. Tests are fast and completely isolated, and they break whenever the internal design changes.
- **A unit is a unit of behaviour**, which may run through several classes. Only what is slow or outside the process gets replaced.

Martin Fowler, borrowing Jay Fields' terms, calls these **solitary** and **sociable** tests. A solitary test isolates the class from all its collaborators; a sociable test lets it use the real ones.

There is no settled definition, and saying so is part of a good answer. What follows is the strategy these notes use, stated as a choice rather than as a fact: **treat the unit as a unit of behaviour, and replace only what is slow or controlled by somebody else.** Real collaborators inside our own process; doubles for the database, the network, the clock and the file system. That keeps tests fast without welding them to the current class structure.

Other teams draw it elsewhere and are not wrong to. A team that treats each class as the unit gets more precise failures and pays for them in tests that break on restructuring. What matters in an interview is being able to name the trade rather than assert a definition.

```

class OrderServiceTest {

    // Real. In-process, fast, and part of the behaviour being described.
    private final PricingCalculator pricing = new PricingCalculator();

    // Doubled. Its real implementation reaches an external dependency.
    private final OrderRepository repository = mock(OrderRepository.class);

    private final OrderService service = new OrderService(pricing, repository);

    @Test
    void create_appliesTheGoldTierDiscount() {
        var command = new CreateOrderCommand("C1", Tier.GOLD, new
BigDecimal("100.00"));

        Order order = service.create(command);

        // Asserting the outcome, not that the calculator was called.
        assertThat(order.total()).isEqualToComparingTo("90.00");
    }
}

```

Doubling `PricingCalculator` here would buy nothing and cost something: the test would then pass even if the calculator were wrong, and it would need rewriting if the discount moved into a different collaborator.

7. Structure and naming

Arrange, Act, Assert gives a test three blocks with one job each:

Block	Job
Arrange	Put the world into the state the behaviour needs
Act	Perform the one action under test
Assert	Check the outcome

If the Act block contains more than one call, the test is probably describing more than one behaviour and should be two tests. **Given, When, Then** is the same shape in the vocabulary of BDD, Behaviour Driven Development.

Naming decides whether a failure is useful. `testOrder1` tells a person reading a broken build nothing at all. A name that states the condition and the expected result tells them whether it matters before they open the file:

```

@Test
void create_whenCustomerHasNoPaymentMethod_rejectsTheOrder() { ... }

```

JUnit 5's `@DisplayName` carries a full sentence where a method name would be unwieldy, and `@Nested` groups the cases that share a setup.

Assertion library matters more than it looks, because the failure message is what somebody reads when the build breaks and they have no context:

```
// Fails with: expected: <true> but was: <false>
assertTrue(order.total().equals(new BigDecimal("42.00")));

// Fails with: expected: 42.00 but was: 37.50
assertThat(order.total()).isEqualByComparingTo("42.00");
```

The second message ends the investigation; the first one starts it. AssertJ also handles the `BigDecimal` trap in that example, where `equals` compares scale and `2.0` is therefore not equal to `2.00`.

8. Parameterized and property-based tests

When one behaviour holds across many inputs, a parameterized test states it once:

```
@ParameterizedTest
@CsvSource({
    "100.00, GOLD, 90.00",
    "100.00, SILVER, 95.00",
    "100.00, NONE, 100.00"
})
void discount_variesByCustomerTier(BigDecimal total, Tier tier, BigDecimal expected)
{
    assertThat(discounts.apply(total, tier)).isEqualByComparingTo(expected);
}
```

Sources: `@ValueSource` for one simple argument, `@CsvSource` for several, `@MethodSource` for objects a comma-separated line cannot express, `@EnumSource` to cover every constant, and `@NullAndEmptySource` for the two arguments most often forgotten.

The caution: if each case needs a different assertion, it is several tests sharing one name, and splitting them makes each failure specific again.

Property-based testing changes what the test asserts:

Style	The claim being made
Example-based	"For these three inputs, expect these three outputs"
Property-based	"For every valid input, this must hold"

Rather than listing examples, we state a property and let the tool generate the inputs:

```
@Property
void reversingTwiceReturnsTheOriginal(@ForAll List<String> items) {
    assertThat(reverse(reverse(items))).isEqualTo(items);
}
```

jqwik is the usual Java tool. Its real value is **shrinking**: when it finds a failure it reduces the input to the smallest case that still fails, so a crash on a list of 300 elements arrives as a list of two. Properties worth reaching for are round trips such as serialise then deserialise, invariants such as a total never going negative, and comparisons against a slower but obviously correct implementation.

9. The five kinds of double

Gerard Meszaros' taxonomy. Interviewers ask for it specifically because almost everybody says "mock" for all five, and being precise here is cheap to learn and immediately noticeable.

Double	What it is
Dummy	Passed only to satisfy a signature. Never actually used
Stub	Returns prepared answers. Asks nothing about how it was called
Spy	Records how it was called, so the test can inspect that afterwards
Mock	In the classic taxonomy, set up in advance with the calls it expects, and fails the test if they do not arrive
Fake	A real working implementation, simplified. An in-memory repository is the standard example

The distinction that matters, and the one the question is really testing:

A stub answers questions. A mock checks that a conversation happened.

Stubs support **state verification**, where the assertion is about the result. Mocks support **interaction verification**, where the assertion is about the calls.

In Mockito the same object plays both roles depending on how the test uses it, which is why the words get muddled:

```
// Used as a stub: supplying an answer so the code under test can proceed.
when(repository.findById("A1")).thenReturn(Optional.of(order));

// Used as a mock: the assertion is that this interaction occurred.
verify(notifier).orderConfirmed("A1");
```

@Spy wraps a real instance so most calls execute normally and selected ones are overridden. It is genuinely useful for getting legacy code under test, and in new code it usually means the class is doing two things that want separating.

Fakes are the underused one. An in-memory implementation of a repository interface gives fast tests without stubbing method after method, and unlike a stub it can enforce the real invariants, so a test that saves a duplicate key fails the way production would.

10. The two schools

Two traditions, and knowing that both exist is worth more in an interview than defending either.

	Classicist, also called Detroit or Chicago	Mockist, also called London
Assert on	The resulting state	The interactions that took place
Collaborators	Real where practical	Doubled
Survives refactoring	Well. Internal restructuring does not break it	Poorly. Tests know how the code collaborates
Failure precision	Lower. One fault can fail many tests	Higher. The failing test names the class
Effect on design	Less direct	Drives interfaces out, working from the outside in

Fowler's "Mocks Aren't Stubs" is the reference if the conversation goes deeper.

The position worth stating: **default to the classicist style, and verify interactions where the interaction is the behaviour.** Sending an email, publishing an event, charging a card and writing to an audit log have no state for the test to inspect. The call is the observable effect, so asserting on the call is not coupling to implementation, it is the only assertion available.

11. When not to use a double

Every double is an assumption about how the real thing behaves, and assumptions drift while nothing fails.

- **A value object.** Construct it. A doubled `Money` is more code and less truth than `new Money("42.00")`.
- **The class under test.** Partially doubling the system under test means the test exercises something that does not exist in production.
- **Anything a fake would serve better**, which is usually the case once a stub needs more than two or three methods set up.
- **A type we do not own**, where it can be avoided. This is the one with a real principle behind it.

Doubling a third-party client encodes our belief about a library we do not control. When the library changes behaviour, the double keeps returning what we assumed, and the test keeps passing while production breaks. The remedy is to wrap it behind an interface we define, double that, and cover the one adapter that touches the real library with an integration test. That way the double asserts a contract we own, and there is exactly one place where a real change surfaces.

12. Integration tests

The most overloaded term in testing. It might mean two classes together, a service with its database, or several deployed services talking to one another. It is worth defining before answering a question about it, and a definition that holds up is:

A test that verifies two or more components work correctly together, very often across a boundary that carries real risk: a database, a message broker, another service's API, the file system, or the framework itself.

Both halves matter. Several of our own classes running together is legitimately an integration test, and so is one class against a real database. The reason the boundary gets emphasised is that it is where the surprises are, since those are the parts whose behaviour we did not write and cannot infer from our own code.

Teams draw the line between unit and integration in different places, so the useful move in an interview is to say what we mean before answering. These notes use the strategy from section 6, which puts doubles at the process boundary, and calls a test integration when it stops doubling something on the far side of that.

Spring Boot test slices load a focused subset of the application context for one layer:

Annotation	Loads	Suits
<code>@WebMvcTest</code>	Controllers, serialisation, validation, error handling	Request and response behaviour, with the service layer doubled
<code>@DataJpaTest</code>	Repositories, entity mapping and datasource configuration	Queries and mapping. It substitutes an embedded database by default, so point it at the real one to get the behaviour that matters
<code>@SpringBootTest</code>	The whole context	The wiring itself. Slowest, so use it least

Testcontainers is the piece worth knowing by name, and it is what turns the default above into something worth trusting. Rather than accepting the embedded database the slice substitutes, it starts the real one in a container for the test:

```

@Testcontainers
@SpringBootTest
class OrderRepositoryTest {

    @Container
    @ServiceConnection // Spring Boot points the datasource at
    it
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>
("postgres:16");

    @Autowired
    private OrderRepository repository;

    @Test
    void findsByCustomerAcrossPages() { ... }
}

```

Why this matters more than it sounds: substituting H2 for PostgreSQL produces tests that pass while production fails, and they differ on exactly the things worth testing — types, constraint behaviour, locking, upsert syntax, and how the query planner treats an index. A test against a different database is testing a different system.

Starting a container is not free, so share one where it makes sense rather than starting a fresh one per test. The `static` field above gives one instance for the whole class, and larger suites go further by sharing across classes. Getting that lifecycle right is what keeps the cost of real dependencies tolerable.

Testing a REST API is the most common integration test in a backend service, and it has a checklist worth having ready, because interviewers ask for it directly:

Check	What to assert
Status codes	The right one per case. 201 on successful creation, with <code>Location</code> where the API exposes the created resource's URL, and a deliberate choice between 404 and 200 with an empty body
Response body	Shape, field names, types, and that nothing internal leaks
Validation	Invalid input returns whichever status the API has defined for it, commonly 400 or 422, with an error that names the field
Errors	A consistent error shape across every endpoint, and no stack trace in it
Authentication and authorisation	Unauthenticated gets 401, authenticated but not permitted gets 403. These are different tests
Boundaries	Empty, one, maximum, null, and the longest string the field allows
Idempotency	Where the endpoint claims it, repeat the call and assert one effect

@WebMvcTest covers most of that quickly with the service layer doubled. The API note covers the design side of these, including status code choice and error format, so this is deliberately only the testing half.

13. Contract testing

The problem this solves is specific, and being able to state it is most of the answer.

Service A calls service B. A's own tests double B, so they pass no matter what B actually does today. An end to end test would catch a mismatch, but it needs both services deployed, it is slow, and it only runs late. So a breaking change in B reaches production before anybody learns that A depended on the old behaviour.

Consumer-driven contract testing splits that check in two so neither side needs the other running:

1. The **consumer's** tests run against a local double, and the tool records what they assumed: the requests made, and the responses expected back. That recording is the **contract**.
2. The **provider** replays the contract against itself in its own build, and fails if it no longer satisfies it.

The provider now learns at build time that a change breaks a named consumer, rather than learning it from an incident. Pact is the usual tool for the consumer-first direction; Spring Cloud Contract approaches the same problem starting from the provider.

Where it fits: it can replace a large class of cross-service compatibility checks that would otherwise be written as end to end tests, once there are enough services that deploying all of them for a test has become the bottleneck. It reduces how many end to end combinations are needed rather than removing the layer.

The decision is about how independently the services move, not about who owns them. Two services released together by one team rarely justify the machinery, because a single build already checks both. The same two services justify it as soon as they deploy on separate schedules, or acquire consumers outside the team, or carry compatibility guarantees somebody depends on.

What it does not do: contract testing verifies the shape of the conversation, not that the business behaviour is correct. Both sides still need their own tests.

14. End to end tests

Exercising the whole system the way a user does. The highest-level tests available, running through a real user-facing path, and the most expensive in every sense: slow, needing a deployed environment, failing for reasons unrelated to the change, and reporting failures that rarely say where the fault is. Realistic execution and useful diagnosis are different things, and this layer trades the second for the first.

They earn their place for a **small, deliberately chosen set of the most important journeys** — signing up, checking out, taking a payment. The ones where being broken is a business emergency rather than a defect.

The failure mode is worth describing because it happens to almost every team. Each production incident produces a new end to end test, on the reasonable-sounding principle that this must never happen again. Nothing ever removes one. Eventually the suite runs for ninety minutes and a good share of its failures are environmental, so people re-run it until it goes green. At that point the suite reports nothing at all, including the real failures.

Two things prevent it: keep the number small on purpose and defend that number, and treat a flaky end to end test as a broken test rather than a fact of life.

For load, stress and soak testing, which are a different activity with different goals, see the observability note rather than treating them as part of this layer.

15. Test driven development

TDD, Test Driven Development, is the cycle **red, green, refactor**:

1. **Red.** Write a failing test for the next small piece of behaviour, and watch it fail.
2. **Green.** Write the least code that makes it pass.
3. **Refactor.** Improve the design while the test holds the behaviour in place.

Watching it fail is a real step, not ceremony. A test never observed failing might be asserting nothing, might be testing the wrong object, or might not be running at all.

What it actually provides:

- **Design pressure applied continuously.** Writing the test first means using the interface before building it, and awkward interfaces become obvious immediately rather than after they have callers.
- **Useful coverage as a by-product** when it is practised well, which is a far better route to it than a coverage target. TDD does not guarantee good tests on its own; it just makes writing them the default rather than an afterthought.
- **Short steps**, which keeps debugging short, because the change that broke it was made in the last few minutes.

The honest limits, which make the answer more convincing rather than less:

- TDD is a **design discipline** at least as much as a testing one. Its main benefit is the pressure on the interface, and a team that gets that pressure another way loses less by not practising it.
- It fits best **where the behaviour can be stated clearly enough to drive a test.** TDD can discover an interface incrementally, so the difficulty is not that the design must be known up front. It is that in genuinely exploratory work, where the requirement is still moving, tests written early get discarded as understanding changes.
- It has a genuine learning curve, and done half-heartedly it produces brittle tests that couple to implementation, which is worse than writing them afterwards.

Writing tests after the code is not a failure. Not writing them is.

16. Testing time and asynchronous code

Two sources of flakiness with the same underlying fix: make the thing that varies a dependency the test controls.

Time. Code that calls `Instant.now()` cannot be tested without waiting for the clock:

```
// Untestable without either waiting or changing the system clock.
if (Instant.now().isAfter(order.expiresAt())) { ... }

// Testable: the clock is injected, so the test decides what time it is.
private final Clock clock;

if (clock.instant().isAfter(order.expiresAt())) { ... }
```

```
var clock = Clock.fixed(Instant.parse("2026-03-01T00:00:00Z"), ZoneOffset.UTC);
```

`java.time.Clock` exists for this. It also makes the awkward cases testable at all: month ends, leap days, and the hour that daylight saving removes.

Asynchrony. `Thread.sleep` in a test is simultaneously too short and too long — long enough to slow every run, short enough to fail on a loaded build machine:

```
// Flaky and slow at the same time.
service.submitAsync(order);
Thread.sleep(2000);
assertThat(repository.findById("A1")).isPresent();

// Polls, so it returns as soon as the condition holds and fails only if it never does.
service.submitAsync(order);
await().atMost(Duration.ofSeconds(5))
    .untilAsserted(() -> assertThat(repository.findById("A1")).isPresent());
```

Awaitility is the library. Better still where the design allows it: **remove the asynchrony from the test** by injecting an executor that runs on the calling thread, so the logic is exercised deterministically, and cover the threading itself in a separate, smaller test.

17. Test data

Long setup blocks are where readable tests go wrong, because the reader cannot tell which of the fifteen fields the behaviour actually depends on.

Object mother. A factory of named, meaningful examples:

```
Order order = Orders.paidOrderFor(customer);
```

Test data builder. A fluent builder with sensible defaults, where the test names only what it cares about:

```
Order order = anOrder().withTotal("42.00").withStatus(PAID).build();
```

Which to reach for depends on the shape of the domain rather than taste. A **builder** suits a domain where tests need many small variations, because each test names exactly the fields its behaviour depends on and a reader knows everything unstated is irrelevant. An **object mother** is simpler where there is a small set of canonical scenarios that recur, and it reads better at the call site. The failure mode of the mother is accumulating near-identical variants as cases multiply; the failure mode of the builder is setup noise in tests that only wanted the standard case. Many codebases use both, with the mother built on top of the builder.

Shared mutable fixtures are the trap. A large object graph built in `@BeforeEach` and used by every test in the class means each test depends on setup it did not ask for. Changing it to suit one test breaks others, so it only ever grows, and the tests quietly become order-dependent. Build what the test needs inside the test, and share the builder rather than the built object.

18. Working with legacy code

Michael Feathers defines legacy code as code without tests, which is deliberately provocative and useful in an interview because it moves the conversation from age to risk.

The difficulty is circular: we want to change the code, we need tests to change it safely, and we cannot write tests because the code is not testable. Two tools break the circle.

Characterization tests. Rather than asserting what the code should do, assert what it currently does. Run it, capture the output, and make that output the expected value. The result is not a specification and may well encode existing bugs. That is acceptable, because its job is not to say the behaviour is right, but to **detect that the behaviour changed**, which is exactly what makes refactoring safe.

Seams. A seam is a place where behaviour can be changed without editing the code at that point. Extracting a hard-coded dependency into a constructor parameter creates one, and it is usually the smallest possible change that makes a class testable:

```
// No seam: the dependency is created inside, so a test cannot replace it.
public class OrderService {
    private final PaymentGateway gateway = new StripeGateway();
}

// Seam: the dependency arrives from outside, so a test can supply its own.
public class OrderService {
    private final PaymentGateway gateway;

    public OrderService(PaymentGateway gateway) {
        this.gateway = gateway;
    }
}
```

The order of work matters, and stating it is what makes the answer sound like experience: **find a seam, get the existing behaviour under a characterization test, then refactor, then improve the tests**. Refactoring first is the mistake, because there is nothing to say whether the refactoring preserved anything.

19. Coverage

Line coverage is the proportion of lines executed by the suite. **Branch coverage** is the proportion of branches taken. Branch coverage usually carries more information, because a line containing a condition counts as covered when only one outcome has ever been tried. Neither is sufficient: branch coverage still misses combinations of conditions, and data-dependent faults such as an empty list or a boundary value show as covered when the interesting input was never supplied.

The limitation that governs everything else about it:

Coverage measures execution, not verification

A test that calls a method and asserts nothing produces full coverage of it.

```
@Test
void process() {
    service.process(order);    // 100% coverage of process(). Verifies nothing.
}
```

So the signal only runs one way: **low coverage reliably shows code nobody tested; high coverage does not show that anything is tested.**

Which is why making coverage a target makes it worse. The cheapest way to raise the number is to write tests without assertions, so the measure rises while the thing it was standing in for does not. This is the general effect where a measure adopted as a target stops being a good measure, known as Goodhart's law.

What to do instead: read the uncovered lines rather than the percentage, because the list of what nobody tested is genuinely useful. Treat a sudden drop as worth a conversation. And get new code tested through review, where a person can judge whether the assertions mean anything, which no threshold can.

20. Mutation testing

Answers the question coverage cannot: **are these tests checking anything?**

The method is direct. Change the code deliberately — invert a condition, alter a boundary from `<` to `<=`, replace a return value, remove a method call — and run the suite against each altered version. Each alteration is a **mutant**. If the tests still pass, that mutant **survived**, which means the suite cannot detect that fault.

The **mutation score** is the proportion of mutants killed, and unlike coverage it is a measurement of the tests rather than of which lines ran.

```
// Covered 100% by a test asserting only that the result is positive.
return price.multiply(new BigDecimal("0.9"));

// Mutant: 0.9 becomes 0.8. Still positive, so the test still passes.
// The mutant survives, and the test is worth nothing.
```

PIT, also called Pitest, is the Java tool.

The cost is real: conceptually it runs the suite once per mutant, so it is far slower than a normal build. Tools reduce that considerably by running only the tests that reach the mutated line and by analysing incrementally, and even so it is normally pointed at changed code or run on a schedule rather than on every commit.

Worth raising in an interview even briefly, because mentioning it demonstrates understanding that coverage is a weak signal, which is the actual point of the coverage question.

21. Flaky tests

A flaky test passes and fails without the code changing. It is more damaging than a failing test, because a failing test gets fixed and a flaky one gets re-run.

The damage lands on the whole suite rather than that one test. Once re-running until green is normal behaviour, genuine failures get re-run too, and the suite has stopped being able to tell anybody anything.

Cause	Fix
<code>Thread.sleep</code> and races	Poll for the condition, or remove the asynchrony from the test
Shared mutable state between tests	Reset between tests, or build fresh state per test
Dependence on execution order	Never rely on order. Run in random order deliberately to find these

Cause	Fix
Real dates and times	Inject a <code>Clock</code> . Watch month ends, leap days and daylight saving
Unseeded random data	Seed it, and print the seed on failure so the case can be reproduced
Real network calls	Double them, or run the dependency in a container
Fixed ports and shared resources	Allocate dynamically per test run
Ambient time zone and locale	Set both explicitly rather than inheriting the machine's
Tests running in parallel over one database	Isolate per test, by schema, by container, or by transaction rollback

The policy that works is **quarantine, then fix or delete**, with a time limit on the quarantine. Moving a flaky test out of the main suite stops it damaging trust while keeping it visible, and a test still quarantined a month later was not going to be fixed.

The policy that does not work is automatic retries, however tempting. Retrying makes the symptom disappear while hiding real intermittent failures.

Two different things look identical

Flakiness caused by the test: a sleep, shared state, an unseeded random value, an assumed order. The production code is fine and the test is wrong.

Flakiness caused by the code: a genuine race, a missing lock, an assumption about ordering that the system does not guarantee. The test is right and it has found something.

They present the same way, which is exactly why retrying until green is dangerous: it discards the second kind along with the first. Establish which one it is before deciding what to do, and treat a test that fails intermittently around concurrency as a suspected production bug until shown otherwise.

22. Interview questions

As in the other notes, each question has an **opening** of roughly thirty to forty seconds, whose last sentence points at the deeper material, and a **follow-up** to use when the interviewer takes it. Saying both at once spends material that would have earned a second question.

"What is the difference between a mock and a stub?"

A stub supplies prepared answers so the code under test can proceed, and it makes no claim about how it was called. A mock is set up in advance with the calls it expects and fails the test if they do not arrive. So a stub supports asserting on the resulting state, and a mock supports asserting that a particular interaction happened. The confusion is mostly Mockito's doing, because the same object plays either role depending on how the test uses it.

If they go deeper:

In Mockito, when I write `when(...).thenReturn(...)` I am using it as a stub, and the moment I write `verify(...)` I am using it as a mock. The full taxonomy is Meszaros': dummy, stub, spy, mock and fake. The one I think is most underused is the fake, a real but simplified implementation such as an in-memory repository, because it avoids stubbing method after method and it can enforce the real invariants, so a test that saves a duplicate key fails the way production would rather than passing.

"What is the test pyramid, and do you agree with it?"

The shape is a wide base of fast unit tests, fewer integration tests, and very few end to end tests, and the reasoning is about cost and feedback rather than virtue: a unit test fails in milliseconds and names one class, an end to end test fails in minutes and tells us something somewhere is wrong. I agree with the principle, which is to use the fastest test that can catch the fault. I would not defend the exact shape, because there are two fair criticisms of it.

If they go deeper:

The first is that its layers are undefined, so it recommends more of something it has not specified, and unit and integration mean different things in different teams. The second is that the right shape depends on the system. The testing trophy makes integration the largest layer, and for a service that is mostly orchestration that is straightforwardly correct, because the complexity is in how the pieces combine rather than inside any one of them and there is little to unit test. What I am most confident about is the inverted shape, many end to end tests over few unit tests. That one tends to produce slow suites, weak failure isolation, and eventually enough flakiness that people re-run failures until they pass.

"How do you decide what a unit is?"

I treat the unit as a unit of behaviour rather than a class, and I put the boundary at the process. Real collaborators inside our own process; doubles for the database, the network, the clock and the file system. So a test of an order service would use the real pricing calculator and double the repository. That keeps tests fast without coupling them to the current class structure, which is the trade the alternative loses.

If they go deeper:

The alternative is to treat every class as the unit and double all its collaborators, which Fowler calls a solitary test as against a sociable one. It gives very precise failures, and the cost is that the tests know the internal design, so any restructuring breaks them even when behaviour did not change. My test is whether a pure refactoring can break the test: if it can, the test was testing implementation. Doubling the pricing calculator in that example would also mean the test passes when the calculator is wrong, which is a strange thing for a test to do.

"Take a typical backend feature. What would you test at each level?"

I would work from risk rather than from a quota. Take adding a discount to an order: the discount calculation itself is high impact and has real branches, so that gets unit tests across every tier and every boundary, including zero and the maximum. The repository and the query get an integration test against the real database in a container, because that is where mapping and constraint behaviour actually differ. The controller gets a slice test for status codes, validation and the error shape. And then one end to end test for the whole purchase, because checkout being broken is a business emergency.

If they go deeper:

What I would not add is a test for the entity-to-response mapping when it has no logic in it, because the controller test already covers it and a dedicated one just restates the code. The rule I use is to push each check to the lowest level that would actually have caught the fault, so a discount rounding error should fail a unit test rather than a checkout journey. If we find ourselves relying on the end to end test to catch a calculation bug, the coverage is at the wrong level and the feedback will arrive minutes later than it needed to.

"How do you decide what not to test?"

By treating tests as having a cost, which is the part people skip. Each one is written once, run thousands of times, maintained forever, and adds friction to every future change, so the question is where the effort is worth spending rather than whether to test. I let depth follow risk: business impact, complexity, how often the code changes, how much depends on it, and how confident we are that we understood the requirement in the first place.

If they go deeper:

Concretely I would not write dedicated tests for getters and plain data holders, for mapping with no branches in it, or for the framework itself, since testing that a query annotation works is testing somebody else's product. Private methods I test through the public surface, and a private method that really seems to want its own test is usually a separate class trying to exist. The general form is to test behaviour that can be wrong in a way that matters, and to be willing to say out loud that some code does not qualify. A suite where every class has a test file regardless of what is in it is usually one where nobody chose anything.

"Is 100% code coverage a good target?"

No, and mainly because coverage measures execution rather than verification. A test that calls a method and asserts nothing reports full coverage of it. So the signal only runs one way: low coverage reliably shows code nobody tested, and high coverage does not show that anything is actually checked. Setting it as a target makes it worse, because the cheapest way to raise the number is to write tests without assertions.

If they go deeper:

That is Goodhart's law in miniature, a measure adopted as a target ceasing to be a good measure. What I do find useful is reading the uncovered lines rather than the percentage, since a list of what nobody tested is real information, and treating a sudden drop as worth a conversation. If the underlying question is whether the tests are any good, the tool that answers it is mutation testing: change the code deliberately, invert a condition or move a boundary, and see whether any test notices. PIT does this for Java. It is slow enough that it is usually pointed at changed code, but it measures the tests rather than which lines ran.

"How do you test something that depends on the current time, or that runs asynchronously?"

Both come down to making the thing that varies a dependency the test controls. For time, inject a `java.time.Clock` instead of calling `Instant.now()`, so the test decides what time it is. That also makes the awkward cases testable at all, month ends, leap days, and the hour daylight saving removes. For asynchrony, the thing to avoid is `Thread.sleep`, which manages to be too short and too long at once.

If they go deeper:

Too long because it slows every run, and too short because it fails on a loaded build machine, so it is both slow and flaky. The direct fix is to poll for the condition with a timeout, which is what Awaitility does, so the test returns as soon as the condition holds and fails only if it genuinely never does. Where the design allows it, I prefer to remove the asynchrony from the test entirely by injecting an executor that runs on the calling thread, then cover the threading itself in a separate and much smaller test. That turns one unreliable test into two deterministic ones.

"How do you handle a flaky test?"

I treat it as more urgent than a failing test, which surprises people. A failing test gets fixed; a flaky one gets re-run, and once re-running until green is normal, real failures get re-run too and the suite has stopped telling anybody anything. So the policy is quarantine, then fix or delete with a time limit, because moving it out of the main suite stops the damage to trust while keeping it visible. What I would argue against is automatic retries.

If they go deeper:

Retrying hides the symptom, and intermittent failures are often the most interesting faults in the system, because a race condition in production code presents exactly like a flaky test. The causes are a short list: sleeps and races, shared state between tests, dependence on execution order, real dates, unseeded random data, real network calls, fixed ports, and the ambient time zone. Running the suite in random order deliberately is a cheap way to find the order-dependent ones, and they are the class most likely to be hiding a real bug rather than a test defect.

"What is contract testing and when would you use it?"

It solves a specific gap. When our service calls another one, our own tests double it, so they pass regardless of what that service actually does today. An end to end test would catch a mismatch but needs everything deployed and runs late. Consumer-driven contract testing splits the check: the consumer's tests record what they assumed about the provider, that recording becomes the contract, and the provider replays it in its own build and fails if it no longer satisfies it. Neither side needs the other running.

If they go deeper:

The payoff is that the provider finds out at build time that a change breaks a named consumer, rather than finding out from an incident. Pact is the usual tool for the consumer-first direction, and Spring Cloud Contract approaches it from the provider side. For two tightly coordinated services released together by one team I would usually skip the machinery, since a single build already checks both, and I would reach for it once they start deploying on separate schedules, or pick up consumers outside the team, or carry a compatibility guarantee somebody relies on. The factor is how independently they move rather than who owns them. The limit worth stating is that it verifies the shape of the conversation rather than the business behaviour, so both sides still need their own tests.

"How would you test a REST API?"

At the level below end to end wherever possible, so with a slice test that loads the web layer and doubles the service beneath it. The checks I would want are the status code per case, so 201 on a successful create with a `Location` header where the API exposes the new resource's URL, and a deliberate choice between 404 and an empty 200; the response shape, with nothing internal leaking; validation returning whatever status the API has defined for it, commonly 400 or 422, with an error that names the field; and a consistent error shape across every endpoint with no stack trace in it. Then authentication and authorisation as separate tests, because they are separate failures.

If they go deeper:

Unauthenticated should be 401 and authenticated but not permitted should be 403, and a test that only covers the first leaves the more dangerous case unchecked. Beyond that I would cover boundaries deliberately: empty, one, the maximum, null, and the longest string the field accepts. Where an endpoint claims to be idempotent I would call it twice and assert one effect, since that claim is easy to make and easy to break. What the slice test cannot tell me is whether the API still matches what its callers expect, which is where contract testing comes in rather than adding more end to end coverage.

"Do you practise test driven development?"

Where it fits, and I would rather describe where that is than claim it is always right. The cycle is red, green, refactor, and watching the test fail first is a real step rather than ceremony, because a test never seen failing might be asserting nothing. What I get from it is mostly design pressure: writing the test first means using the interface before building it, so an awkward interface becomes obvious immediately instead of after it has callers. Coverage arrives as a by-product, which is a much better route to it than a target.

If they go deeper:

The honest limits are that it is a design discipline at least as much as a testing one, so a team getting that pressure another way loses less by not doing it; that it fits best where the behaviour can be stated clearly enough to drive a test, and I would be careful there, because TDD can discover an interface incrementally, so the problem is not that the design must be known up front but that in genuinely exploratory work the requirement is still moving and early tests get discarded; and that done half-heartedly it produces tests coupled to implementation, which is worse than writing them afterwards. Writing tests after the code is not a failure. Not writing them is.

23. Common mistakes

- Asserting on how the code works rather than what it does, so every refactoring breaks the suite and the team learns to treat failures as noise.
- Doubling a collaborator that is fast and in-process, which makes the test pass even when the collaborator is wrong.
- Doubling a type we do not own, so the double keeps returning what we assumed after the real library changed.
- Calling everything a mock, when a stub, a fake or a real object was what the test needed.
- Stubbing method after method where an in-memory fake would be shorter and would enforce the real invariants.
- Substituting H2 for the production database, so tests pass on exactly the behaviour that differs: types, constraints, locking and upserts.
- Naming tests `test1`, so a failure in a build says nothing about whether it matters.
- Using `assertTrue` on a comparison, so the failure reports `expected true, was false` instead of the two values.
- Comparing `BigDecimal` with `equals`, where `2.0` is not equal to `2.00` because the scale differs.
- More than one action in the Act block, which makes the failure ambiguous about which one broke.
- `Thread.sleep` to wait for asynchronous work, which is slow on every run and flaky on a loaded one.
- Calling `Instant.now()` in production code, then being unable to test month ends, leap days or expiry at all.
- A large shared fixture in `@BeforeEach`, which only grows and quietly makes tests order-dependent.

- Treating coverage as a target, which rewards tests with no assertions.
- Reading the coverage percentage rather than the list of uncovered lines, which is where the information is.
- Re-running a flaky test until it passes, which hides the intermittent real failures too.
- Adding an end to end test after every incident and never removing one, until the suite takes an hour and nobody trusts it.
- Refactoring legacy code before getting it under a characterization test, so nothing says whether behaviour was preserved.
- Writing a test file per class as a matter of policy, which spends effort by quota rather than by risk and leaves the dangerous code no better covered than the trivial code.
- Testing that the framework works, such as asserting a query annotation returns rows, rather than testing our own query against a real database.
- Covering only the unauthenticated case on a protected endpoint, so 401 is tested and 403, the more dangerous one, is not.

24. Quick reference

Term	Meaning
Test level	How much of the system runs: unit, integration, contract, end to end
Test purpose	Why the test exists: regression, smoke, acceptance, performance, security
Levels against purposes	Independent dimensions. A smoke test can be a unit test; a regression test can sit at any level
Risk-based testing	Depth follows business impact, complexity, change frequency, reach and uncertainty
Test pyramid	Many fast unit tests, fewer integration, very few end to end. A heuristic, not a law
Ice cream cone	The inverted pyramid. Slow, flaky, and eventually ignored
Testing trophy	Integration as the largest layer. Often right for orchestration-heavy services
Solitary against sociable	Collaborators doubled, against collaborators real
Arrange, Act, Assert	Set up, perform one action, check the outcome
Given, When, Then	The same three blocks in BDD vocabulary
Dummy	Passed to satisfy a signature, never used
Stub	Returns prepared answers. Supports state verification
Spy	Records how it was called, for inspection afterwards
Mock	Expects specific calls, and fails when they do not arrive. Supports interaction verification

Term	Meaning
Fake	A simplified working implementation, such as an in-memory repository
Classicist school	Assert on state, keep collaborators real. Survives refactoring
Mockist school	Assert on interactions, double collaborators. Precise failures, brittle under refactoring
Integration test	Verifies two or more components work together, often across a boundary that carries real risk
@WebMvcTest	Controllers, serialisation and validation, without a database
@DataJpaTest	Repositories, mapping and datasource configuration. Substitutes an embedded database unless pointed at a real one
Testcontainers	Runs the real dependency in a container for the test
@ServiceConnection	Points Spring Boot's datasource at the container automatically
Contract testing	Consumer records its assumptions; provider replays them in its own build
Characterization test	Asserts what the code currently does, so change is detectable
Seam	A place where behaviour can be changed without editing the code there
Line against branch coverage	Lines executed, against branches taken. Branch carries more, and neither is sufficient
Mutation testing	Alter the code and see whether any test notices. Measures the tests
Mutant survived	The tests did not detect a deliberately introduced fault
Property-based testing	State a property, let the tool generate inputs and shrink any failure
Shrinking	Reducing a failing input to the smallest case that still fails
Quarantine	Move a flaky test out of the main suite, with a time limit to fix or delete

Symptom	What it usually means
The test broke but behaviour did not change	It was asserting on implementation
A test needs six doubles to set up	The class has six dependencies. A design problem, reported by the test
Tests pass, production fails on the query	The test ran against a different database
Passes alone, fails in the suite	Shared state or order dependence
Fails only in CI	Timing, time zone, locale, or an unseeded random value
Coverage high, bugs still reaching production	Tests execute the code without asserting on it. Try mutation testing
The suite takes an hour	Too much at the end to end layer. Push checks downward

Symptom	What it usually means
Nobody trusts a red build	Flakiness has already cost the suite its value. Quarantine first
Cannot test it without waiting	The clock or an executor is created inside instead of injected
Cannot test it at all	No seam. Extract one dependency into the constructor and start there
A test file for every class regardless	Testing by quota rather than by risk. Ask what could be wrong in a way that matters
Intermittent failure around concurrency	Treat as a suspected production race until shown to be the test's fault